

30 Days SQL Micro Course

Practise Sql Queries 1

Queries using only:

- CREATE TABLE
- INSERT
- SELECT
- WHERE
- ORDER BY
- LIMIT
- UPDATE
- DELETE

Practise link : <https://dbfiddle.uk/vKsjhDAw>

Step 1: CREATE TABLE

Question

Create a table to store employee information.

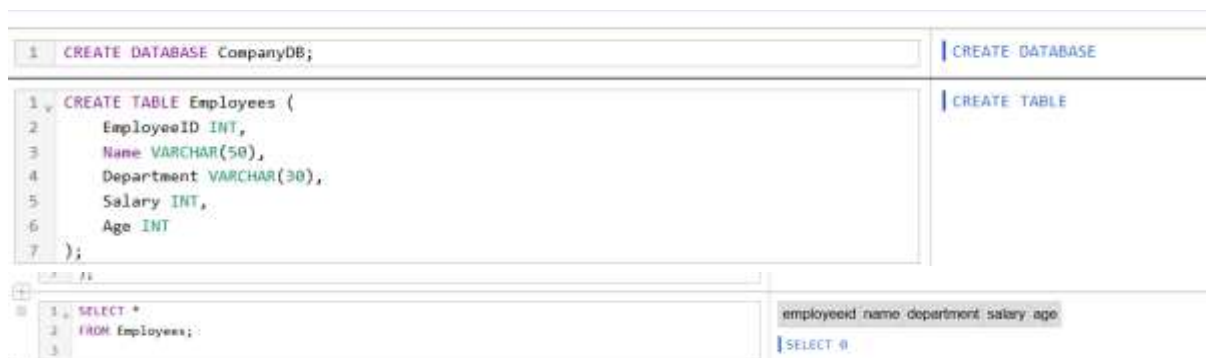
Think

What information do we need?

Column	Data Type
EmployeeID	INT
Name	VARCHAR(50)
Department	VARCHAR(30)
Salary	INT
Age	INT

Query

```
CREATE TABLE Employees (  
    EmployeeID INT,  
    Name VARCHAR(50),  
    Department VARCHAR(30),  
    Salary INT,  
    Age INT  
);
```



What happened?

A table named Employees is created.

Step 2: INSERT DATA

Question

Add employee records.

Query

```
INSERT INTO Employees
```

```
VALUES (1, 'Amit', 'IT', 50000, 25);
```

```
INSERT INTO Employees
```

```
VALUES (2, 'Neha', 'HR', 60000, 28);
```

```
INSERT INTO Employees
```

```
VALUES (3, 'Rahul', 'IT', 70000, 30);
```

INSERT INTO Employees

VALUES (4, 'Priya', 'Sales', 45000, 24);

INSERT INTO Employees

VALUES (5, 'Ravi', 'HR', 55000, 29);

```
1 INSERT INTO Employees
2 VALUES (1, 'Amit', 'IT', 50000, 25);
3
4 INSERT INTO Employees
5 VALUES (2, 'Neha', 'HR', 60000, 28);
6
7 INSERT INTO Employees
8 VALUES (3, 'Rahul', 'IT', 70000, 30);
9
10 INSERT INTO Employees
11 VALUES (4, 'Priya', 'Sales', 45000, 24);
12
13 INSERT INTO Employees
14 VALUES (5, 'Ravi', 'HR', 55000, 29);
```

```
1 INSERT @ 1
2 INSERT @ 1
3 INSERT @ 1
4 INSERT @ 1
5 INSERT @ 1
```

```
1 SELECT *
2 FROM Employees;
```

employeeid	name	department	salary	age
1	Amit	IT	50000	25
2	Neha	HR	60000	28
3	Rahul	IT	70000	30
4	Priya	Sales	45000	24
5	Ravi	HR	55000	29

```
1 SELECT 5;
```

Step 3: SELECT

Question

Show all employees.

Think

Need:

- All columns

Query

SELECT *

FROM Employees;

```
1 SELECT *
2 FROM Employees;
```

employeeid	name	department	salary	age
1	Amit	IT	50000	25
2	Neha	HR	60000	28
3	Rahul	IT	70000	30
4	Priya	Sales	45000	24
5	Ravi	HR	55000	29

```
1 SELECT 5;
```

Step 4: SELECT Specific Columns

Question

Show employee names only.

Think

Need:

- Name column

Query

```
SELECT Name  
FROM Employees;
```

Question

Show Name and Salary.

Query

```
SELECT Name, Salary  
FROM Employees;
```



name
Amit
Neha
Rahul
Priya
Ravi

name	salary
Amit	50000
Neha	60000
Rahul	70000
Priya	45000
Ravi	55000

Step 5: WHERE

Question

Show employees older than 25.

Think

Filter Age.

```
SELECT *  
FROM Employees  
WHERE Age > 25;
```


1. SELECT *	SELECT 3
2. FROM Employees	employeeid name department salary age
3. ORDER BY Salary DESC;	3 Rahul IT 70000 30
	2 Neha HR 60000 28
	5 Ravi HR 55000 29
	1 Amit IT 50000 25
	4 Priya Sales 45000 24
1. SELECT *	SELECT 5
2. FROM Employees	employeeid name department salary age
3. ORDER BY Salary;	4 Priya Sales 45000 24
	1 Amit IT 50000 25
	5 Ravi HR 55000 29
	2 Neha HR 60000 28
	3 Rahul IT 70000 30
	SELECT 5

Step 7: LIMIT

Question

Show first 3 employees.

SELECT *
FROM Employees
LIMIT 3;

1. SELECT *	SELECT 5
2. FROM Employees	employeeid name department salary age
3. LIMIT 3;	1 Amit IT 50000 25
	2 Neha HR 60000 28
	3 Rahul IT 70000 30
	SELECT 5

Step 8: UPDATE

Question

Change Amit's salary to 55000.

UPDATE Employees
SET Salary = 55000
WHERE Name = 'Amit';

Check:

SELECT *
FROM Employees;

1. UPDATE Employees	UPDATE 1
2. SET Salary = 55000	employeeid name department salary age
3. WHERE Name = 'Amit';	2 Neha HR 60000 28
	3 Rahul IT 70000 30
	4 Priya Sales 45000 24
	5 Ravi HR 55000 29
	1 Amit IT 55000 25
1. SELECT *	SELECT 5
2. FROM Employees;	

Step 9: DELETE

Question

Delete Priya's record.

DELETE FROM Employees
WHERE Name = 'Priya';

Check:

SELECT *
FROM Employees;

The screenshot shows a SQL IDE with two panels. The left panel contains the SQL code: `1 DELETE FROM Employees` and `2 WHERE Name = 'Priya';`. Below it, the code `1 SELECT *` and `2 FROM Employees;` is shown. The right panel displays the result of the `DELETE` statement, showing a table with 4 rows and 5 columns: `employeeid`, `name`, `department`, `salary`, and `age`. The data is as follows:

employeeid	name	department	salary	age
2	Neha	HR	60000	28
3	Rahul	IT	70000	30
5	Ravi	HR	55000	29
1	Amit	IT	55000	25

Solving sql queries: <https://leetcode.com/problems/recyclable-and-low-fat-products/?envType=study-plan-v2&envId=top-sql-50>

The screenshot shows the LeetCode SQL 50 problem interface. The left panel displays the SQL Schema for the `Products` table:

Column Name	Type
product_id	int
low_fats	enum
recyclable	enum

product_id is the primary key (column with unique values) for this table.
low_fats is an ENUM (category) of type ('Y', 'N') where 'Y' means this product is low fat and 'N' means it is not.
recyclable is an ENUM (category) of types ('Y', 'N') where 'Y' means this product is recyclable and 'N' means it is not.

Write a solution to find the ids of products that are both low fat and recyclable.
Return the result table in **any order**.
The result format is in the following example:

Example 1:

The right panel shows the Code editor with the following SQL query:

```
1 # Write your MySQL query statement below
2 SELECT product_id
3 FROM Products
4 WHERE low_fats = 'Y'
5 AND recyclable = 'Y';
```

Below the code editor, the Testcase and Test Result sections are visible. The Testcase shows the Products table with the following data:

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

The Output section shows the result of the query:

product_id
1
3

filter by specific product_ids. Instead, you need to select products where:

- `low_fats = 'Y'`
- `recyclable = 'Y'`

584. Find Customer Referee

Easy Topics Companies Hint

SQL Schema > Pandas Schema >

Table: Customer

Column Name	Type
id	int
name	varchar
referee_id	int

In SQL, id is the primary key column for this table. Each row of this table indicates the id of a customer, their name, and the id of the customer who referred them.

Find the names of the customer that are either:

1. **referred by** any customer with `id != 2`.
2. **not referred by** any customer.

Return the result table in **any order**.

The result format is in the following example.

Solved

Code

```
MySQL
1 # Write your MySQL query statement below
2 SELECT name
3 FROM Customer
4 WHERE referee_id != 2
5      OR referee_id IS NULL;
```

Testcase Test Result

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

Output

name
Will
Jane
Bill
Zack

Include customers whose referee is NOT 2

Also include customers with no referee

SQL 50

Description Editorial Solutions Submissions

SQL Schema > Pandas Schema >

Table: world

Column Name	Type
name	varchar
continent	varchar
area	int
population	int
gdp	bigint

name is the primary key (column with unique values) for this table. Each row of this table gives information about the name of a country, the continent to which it belongs, its area, the population, and its GDP value.

A country is **big** if:

- it has an area of at least three million (i.e., `area >= 3000000`), or
- it has a population of at least twenty-five million (i.e., `population >= 25000000`).

Write a solution to find the name, population, and area of the **big countries**.

Return the result table in **any order**.

The result format is in the following example.

Code

```
MySQL
1 # Write your MySQL query statement below
2 SELECT name, population, area
3 FROM world
4 WHERE area >= 3000000
5      OR population >= 25000000;
```

Testcase Test Result

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28740	2831741	12960000000
Algeria	Africa	2381741	37100000	180601000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20605294	100990000000

Output

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Expected

name	population	area
------	------------	------

- at least 3,000,000 → `>= 3000000`
- more than 3,000,000 → `> 3000000`
- at most 3,000,000 → `<= 3000000`

- less than 3,000,000 → < 3000000

A country is big if:

- it has an area of at least 3,000,000
- OR it has a population of at least 25,000,000

Translate directly to SQL:

area >= 3000000

OR population >= 25000000

The screenshot shows a SQL editor interface with the following components:

- Description:** A table named 'Views' with columns: article_id (int), author_id (int), viewer_id (int), and view_date (date). It notes that there is no primary key and that equal author_id and viewer_id indicate the same person. The task is to find authors who viewed at least one of their own articles, sorted by id in ascending order.
- Code:** A MySQL query: `SELECT DISTINCT author_id AS id FROM Views WHERE author_id = viewer_id ORDER BY id;`
- Test Result:** A table showing the results of the query:

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
- Output:** A table showing the final result:

id
4
7

equal author_id and viewer_id indicate the same person

and

find all the authors that viewed at least one of their own articles

Translate to SQL logic

If an author viewed their own article:

author_id = viewer_id

From the sample:

article_id author_id viewer_id

2 7 7

3 4 4

article_id author_id viewer_id

3 4 4

Authors 7 and 4 viewed their own articles.

Requirements

1. Return column name as id
2. Remove duplicates (DISTINCT)
3. Sort ascending (ORDER BY)

The screenshot shows a SQL editor interface with a problem description on the left and a code editor on the right. The problem description includes a table schema for 'Tweets' and a query requirement to find tweet IDs where the content length is strictly greater than 15. The code editor shows a MySQL query that selects tweet IDs from the 'Tweets' table where the length of the content is greater than 15. The 'Testcase' section shows the expected output for the given input data.

Table: Tweets

Column Name	Type
tweet_id	int
content	varchar

tweet_id is the primary key (column with unique values) for this table.
content consists of alphanumeric characters, '!', or ' ' and no other special characters.
This table contains all the tweets in a social media app.

Write a solution to find the IDs of the invalid tweets. The tweet is invalid if the number of characters used in the content of the tweet is **strictly greater** than 15.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:
Tweets table:

tweet_id	content
1	Let us Code
2	More than fifteen chars are here!

MySQL Code:

```
1 # Write your MySQL query statement below
2 SELECT tweet_id
3 FROM Tweets
4 WHERE LENGTH(content) > 15;
```

Testcase:

Expected:

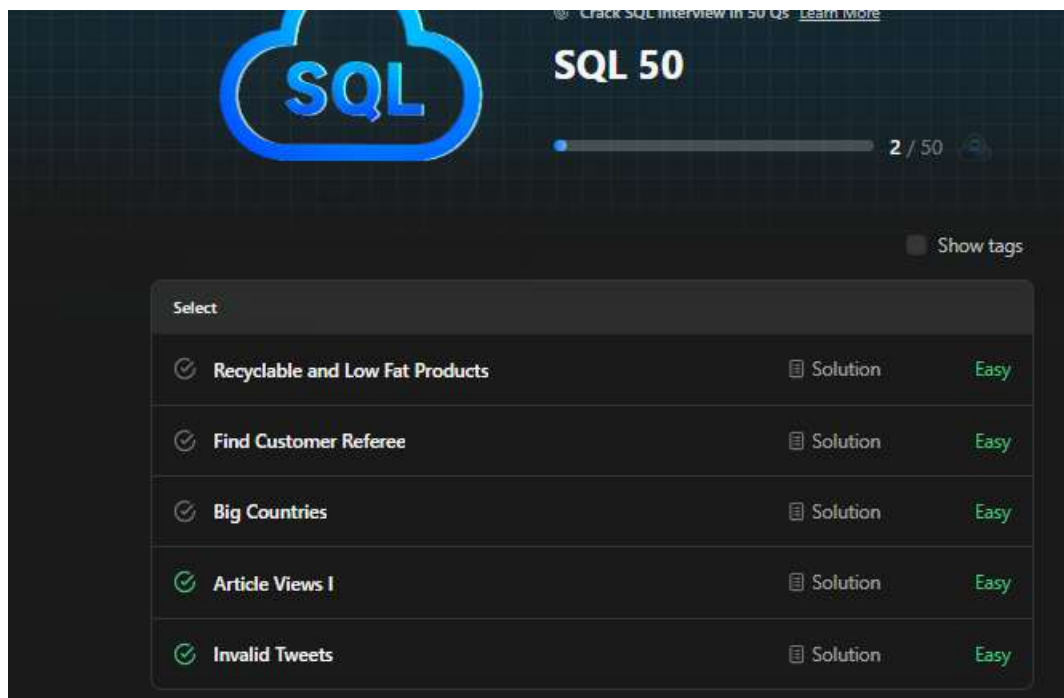
tweet_id	content
2	More than fifteen chars are here!

number of characters used in the content is strictly greater than 15

Translate English → SQL

- "strictly greater than 15" → > 15
- Count characters in a string → LENGTH()
- Useful SQL String Functions

Function	Purpose
LENGTH(col)	Number of characters
UPPER(col)	Uppercase text
LOWER(col)	Lowercase text
CONCAT(a,b)	Join strings



Common Questions → SQL

Need 2 conditions?

WHERE A AND B

Example:

WHERE low_fats='Y' AND recyclable='Y'

Need either condition?

WHERE A OR B

Example:

WHERE area >= 3000000

OR population >= 25000000

Need empty values?

WHERE column IS NULL

Remove duplicate rows?

**SELECT DISTINCT column
FROM table;**

Rename a column?

**SELECT author_id AS id
FROM Views;**

Sort data?

ORDER BY id

Count letters in text?

LENGTH(content)